

CS 409: MP2

https://gemini.google.com/app/2d8b366188d8f7b2

User prompt: Now let's say that you are working in a `tsx` file where you have a function and in that, you have a `return` statement where you are creating the buttons, like how we have discussed above, please tell me where to write the functionality without giving away the final answer. Should that be written inside an attribute?

Response: Yes, the button's action is tied to an attribute—specifically an event handler attribute on the button element (like `onClick`)—but where you place the actual logic depends on how complex that logic is:

1. The Attribute Links the Action

In JSX/TSX, the button communicates with your logic through the `onClick` attribute using curly braces `{}`. This attribute expects a function reference or a callback to run when the user clicks.

2. Where to Write the Actual Logic

You have two primary options:

- **Above the `return` statement (Recommended for anything beyond a one-liner):**
 - You define dedicated handler functions (e.g., one for going previous, one for going next) inside your component body, right before the `return` statement.
 - In the JSX, you simply pass the function's name into the button's `onClick` attribute without invoking it immediately.
 - This keeps your JSX clean, readable, and easy to debug when you need to calculate neighbor indices or call navigation functions.
- **Directly inside the attribute (Inline Arrow Function):**
 - You can write an inline arrow function directly inside the attribute: `onClick={() => /* action here */}`.
 - This is fine for simple, single-expression triggers, but if the logic involves calculations, boundary checks, or multiple steps, writing it inside helper functions above the `return` statement is much cleaner.

User prompt: burlywood -> a color that I am using in CSS. Please give me a light version and a dark version (by 10 %)?

Response: The standard CSS named color `burlywood` corresponds to `#DEB887`, or `hsl(34, 57%, 70%)` in HSL.

Here are the 10% lighter and 10% darker versions using the standard HSL lightness model:

Variant	Hex Code	HSL / RGB Value	Preview Description
Light (+10% Lightness)	#E9D0AF	hsl(34, 57%, 80%) rgb(233, 208, 175)	Soft cream/pale sand
Base (burlywood)	#DEB887	hsl(34, 57%, 70%) rgb(222, 184, 135)	Classic burlywood
Dark (-10% Lightness)	#D3A05F	hsl(34, 57%, 60%) rgb(211, 160, 95)	Warm golden tan

Alternative: Native CSS `color-mix()`

In modern CSS, you can also compute tints and shades directly in your stylesheet without hardcoding hex codes:

```

/* 10% lighter (mixed with 10% white) */
background-color: color-mix(in srgb, burlywood, white 10%); /* Hex equivalent: #E1BF93 */

/* 10% darker (mixed with 10% black) */
background-color: color-mix(in srgb, burlywood, black 10%); /* Hex equivalent: #C8A67A */

```

User prompt: For the button function for previous and next, I made a slight change. Does this make sense: In the `tsx` function, I store the previous data slide and the next data slide info: `const prev_idx = curr_idx === 0 ? images.length - 1 : curr_idx - 1` `const next_idx = curr_idx === images.length - 1 ? 0 : curr_idx + 1` `const prev_img = images[prev_idx]` `const next_img = images[next_idx]` And then using the `Link` type in HTML, I have created the previous and next buttons: `<Link className='prev-button' aria-label='Previous planet' to={'/details/' + encodeURIComponent(prev_img.data[0].nasa_id)}>{'<' }</Link>` Similar for next as well. Does this make sense.

Response: The logic and structure are clean, sound, and align well with what you are building.

A few specific strengths stand out:

- **Clean Wrap-Around Boundary Logic:** Using `curr_idx === 0 ? images.length - 1 : curr_idx - 1` and `curr_idx === images.length - 1 ? 0 : curr_idx + 1` correctly creates an infinite loop through your dataset without going out of bounds.
- **Safe JSX Syntax:** Escaping the arrow as `{' < '}` completely eliminates JSX parser collision errors with the `<` character.
- **URL Encoding:** Adding `encodeURIComponent(...)` is good defensive practice, ensuring any special characters or spaces in raw NASA IDs won't break the URL route.
- **Accessibility:** Including `aria-label='Previous planet'` makes your icon-only navigation accessible to screen readers.

Potential Edge Cases to Watch

1. **Empty Array Check (`images.length === 0`):** If the component renders while data is still loading or if a search returned zero items, `images.length - 1` becomes `-1`, causing `images[-1]` to evaluate to `undefined`. Accessing `prev_img.data[0]` will then throw a `TypeError: Cannot read properties of undefined`.

Fix: Add a quick guard condition before calculating indices:

```

if (!images || images.length === 0 || curr_idx === -1) {
  return <p>Loading or item not found...</p>;
}

```

2. **Single-Item Datasets (`images.length === 1`):** If there is only 1 item in the array, `prev_idx` and `next_idx` will both evaluate to 0, pointing back to the current item. This won't throw an error, but you can optionally hide or disable the buttons if `images.length <= 1`.

User prompt: I don't know if that's a concern because when you open the website. You would first end up in the Search page and if you go to the Gallery page and see that it is empty, it makes sense that the data is being loaded and after a couple of minutes, once the data is finally loaded, then we can click on one of the cards and then it will show us the data slide and then we can happily move to the previous and next button. What do you think because I don't think that it is quite necessary.

Response: That mental model works when everything happens in the ideal sequence, but in web development, users and graders don't always take the "happy path."

Here is why that guard check is practically essential rather than just theoretical:

1. Direct URL Access & Page Refreshes

The project requirement specifically states:

"A user should be able to access the detail route through a specific url."

If a user (or course grader) copies the link `http://localhost:5173/details/PIA21422`, pastes it into a brand-new browser tab, or simply hits **Refresh** (Cmd/Ctrl + R) while on that detail page:

- React's component state resets to empty.
- The API request starts fetching in the background.

- The detail component mounts **before** the fetch finishes.
- At that exact millisecond, `images` is empty (`[]`).
- Without a guard, `prev_img.data[0]` immediately crashes the entire app with a blank white screen before the data ever gets a chance to arrive.

2. Search Results Can Be Empty

If a user searches for an exoplanet name with a typo (e.g., "Keplerr-999"), the search array might have 0 items. If an invalid or non-existent ID ends up in the URL, `curr_idx` will be `-1`.

3. A 2-Line Fix Prevents a Fatal App Crash

Handling this doesn't require complex logic—it just needs an early exit to protect the render:

```
// If data hasn't arrived yet, or the ID isn't found
if (!images.length || curr_idx === -1) {
  return <div className="loading">Loading planet details...</div>;
}
```

Once the fetch resolves and `images` populates, React will automatically re-render the component, your index math will run smoothly, and the buttons will appear without any risk of crashing the page.

User prompt: Hmm. I see the point. I will take a look into this. Thank you.

Response: You're very welcome. That tiny guard pattern is one of those simple defensive additions that saves hours of tracking down elusive white-screen bugs during refreshes.

Take your time testing the direct links and navigation flow, and give a shout whenever you're ready to tackle the next piece.

User prompt: So for this project for an API, a bit of search took place and instead of relying on the ones that were given in the assignment, I stumble across one that I thought is good for the API and for the MP. I finished working using it and I want to confirm whether this is good: <https://science.nasa.gov/wp-json/wp/v2/exoplanet> (endpoint and the one used in the axios function in the App.tsx file) This is from NASA. Is this good for the MP and the website? I got it working.

Response: Using that endpoint is technically interesting, but you need to check a couple of critical course and technical constraints before committing to it for grading.

Here is the breakdown of why:

1. Course Assignment & Rubric Constraints (The Main Risk)

- **Explicit Endpoint / Query Requirements:** In assignments like this MP, the rubric or autograder often expects either the official **NASA Image and Video Library API** (`[https://images-api.nasa.gov/search?q=exoplanet](https://images-api.nasa.gov/search?q=exoplanet)`) or specific fields/data structures provided in the handout.
- **Grading Scripts:** If grading involves an automated test suite or a TA checking network traffic/expected JSON keys against the assignment specification, switching to a completely different API origin can cause unit or integration tests to fail.
- **Rubric Check:** If the prompt stated "you may use any public API" or specifically gave you freedom of choice, you are in the clear. But if it prescribed the Image & Video Library API specifically, stick with the prescribed one (or make sure your adapter layer handles the expected spec).

2. Architecture & Data Shape (`science.nasa.gov/wp-json/wp/v2/exoplanet`)

This specific endpoint is the **WordPress REST API** for NASA's Science Mission Directorate website:

- **Pros:**
 - It returns rich article/post metadata, featured media IDs, rendered HTML content, and exoplanet-related taxonomy.
 - If your adapter maps it cleanly into your internal interface and your UI displays everything smoothly, the frontend engineering itself works.
- **Cons & Gotchas:**

- **Image Access (Pagination & Embedding):** WordPress endpoints often don't include full image URLs in the main item object unless you pass the query parameter `?_embed` (e.g., `.../exoplanet?_embed`). Without `_embed`, it only gives `featured_media: <number>`, which would otherwise require secondary network requests to resolve image links.
- **Rate Limiting & Public Stability:** Unlike official developer portals (e.g., `api.nasa.gov`), internal CMS REST endpoints (`wp-json`) are subject to unexpected schema changes, caching policies, or CORS tightening without advance notice to developers.
- **Pagination Limits:** Standard WordPress endpoints default to 10 items per page unless you pass `?per_page=100`.

Recommended Next Step

If your assignment explicitly permits choosing your own endpoint and you verified that:

1. It delivers images and metadata without CORS or rate-limit issues,
2. It fetches all needed items (e.g., passing `?per_page=...`), and
3. Your transformation adapter cleanly feeds your search, gallery, and detail views,

then it will work visually. However, if the MP spec explicitly required the **NASA Image and Video Library** (`images-api.nasa.gov`), verify with your assignment handout or course staff first so you don't lose points on an API mismatch.

User prompt: Here are the assignment instructions: Requirements Create a single-page React app that lets users interact with the data from one of the following APIs. TMDB Pokemon NASA Marvel Jelly Belly Wiki The Meal DB Art Institute of Chicago More APIs if you want to explore here (please keep it class appropriate) 🙌🤔👉. Note that you may need to create an account and/or acquire an API key for some of the APIs. GitHub won't restrict you from pushing API keys, but may send an email to say some keys are exposed. Because of this, please don't use any paid APIs. The API you are working with may become temporarily unavailable. If/when this happens, it doesn't mean you are blocked from working on the MP. You can mock the data, i.e. create a local hard coded response and use that instead of making the request. You may need to deal with APIs having rate-limiting policies. You can find ways to get around them like caching the results of large and common API calls These are also good opportunities to think about how your app should handle errors. This is regarding the API and I have completely switched from NASA Image API to NASA Exoplanet and it works. Here are some screenshot to prove it. What do you think? I think this is good?

Response: This is a clean, well-executed switch. Because the assignment prompt explicitly lists **"NASA"** broadly as an eligible API source (alongside an open invite to explore class-appropriate APIs), using this NASA endpoint fits the rubric criteria.

Reviewing what you have built across the screenshots:

What Works Well

- **Clear Data Modeling & Attributes:** The detail view displays substantial astronomical data points—Host Star, Planet Type, Discovery Method, Discovery Year, Radius, Mass, and Orbital Period. This fulfills the requirement that a detail view displays the different attributes of the selected item.
- **Navigation & Link Spacing:** The top navigation bar now features clean spacing between **Search** and **Gallery**, resolving the earlier layout issue where the text ran together.
- **Filter Controls & Card Highlighting:** The gallery includes category filters ("Terrestrial", "Super Earth", "Neptune-like", "Gas Giant"), and the burlywood accent styling on the selected card matches the border highlight on the detail view.
- **Detail View Navigation & Cycling:** The `<` and `>` arrow buttons sit on either side of the card, ready to cycle through the dataset, complemented by breadcrumb links back to Search and Gallery.
- **Dataset Volume:** Showing 100 loaded items provides plenty of depth for both filtering in the gallery and searching/sorting.

Quick Final Sanity Checks Before Submitting

1. **Direct Route Access:** In your browser address bar, verify that navigating directly to `/details/toi-286-b` (or refreshing the page while on that screen) loads the planet without flashing a runtime error while the data fetches.
2. **Cycle Consistency:** Ensure clicking `>` and `<` moves sequentially through the active list order (or loops back to the beginning when reaching index 99).
3. **Offline / Fallback Resilience:** The prompt mentions: *"The API you are working with may become temporarily unavailable... You can mock the data."* Ensure your fallback/mock JSON is ready in case the NASA endpoint hits a rate limit or network lag during grading.

User prompt: Okay. Thank you. So am I good for now?

Response: You are in great shape. The core requirements—API integration, list/gallery views, filtering, sorting, dynamic detail routing via `useParams`, and previous/next cycling—are all in place and functioning cleanly on screen.

As long as you've tested a direct page refresh on the detail route and confirmed your mock fallback is wired up, you're ready to wrap this up and push your work.

User prompt: Please help me answer this question for the google form that I need to submit for this class: How much did it cost to use an LLM assistant for this project? Please provide the answer in dollars.

Response: **\$0.00**

You used a standard, free consumer AI chat interface rather than paid, pay-per-token developer API keys.

User prompt: Okay. Thank you for all the help. I will go ahead and get the remaining parts done right away.

Response: You've got this. Good luck wrapping up the final touches and getting your submission in! Feel free to reach back out if anything unexpected pops up during testing.

User prompt: When deploying the website on GitHub and going through one of the data slides, I encountered a 404 error when reloading the website. How to fix it? Please tell me without giving away the code. What should I do?

Response: This is the classic **Single-Page Application (SPA) routing issue on static hosting** like GitHub Pages.

Here is why it happens and the conceptual ways to solve it without touching your core application logic:

Why the 404 Happens

- When you navigate through your app by clicking links, **React Router** handles the transition in the browser using JavaScript without contacting the server.
- When you press **Reload** on a URL like `.../details/toi-286-b`, your browser makes a brand-new HTTP request directly to GitHub's static web servers asking for an actual file named `details/toi-286-b` (or `details/toi-286-b.html`).
- Because GitHub Pages only hosts static files (like your root `index.html`), it cannot find that folder or HTML file on its drive, so it responds with a native **404 Not Found**.

Solution 1: Use **HashRouter** (Quickest & Most Reliable for Class MPs)

The simplest fix that requires zero configuration on GitHub's end is switching your React router implementation:

- **Concept:** Instead of using standard browser paths (managed by `BrowserRouter`), switch your top-level router wrapper to a hash-based router (`HashRouter` from `react-router-dom`).
- **How it works:** Your routes change format slightly to include a hash (e.g., `.../#/details/toi-286-b`).
- **Why it fixes the 404:** Web servers inherently ignore everything in a URL after the `#` symbol. When you reload, the browser asks GitHub Pages strictly for the root `index.html` (which exists and returns HTTP 200). Once the page loads, React Router reads the `#` fragment and renders your detail view seamlessly.

Solution 2: The GitHub Pages 404 Redirect Trick (`404.html`)

If your course rubric strictly forbids hashes in the URL and demands HTML5 path history:

- **Concept:** Provide a custom `404.html` fallback file inside your public build folder.
- **How it works:** When GitHub Pages encounters a route it doesn't recognize on reload, it automatically displays your `404.html` instead of its default error page.
- **The trick:** You place a lightweight script in `404.html` that captures the path the user was trying to visit, redirects back to

`index.html` with that path stored in a query parameter or session storage, and your `index.html` restores the intended route into React Router upon load. (There is a widely used open-source script community known as the *SPA GitHub Pages hack* for this).

Recommendation

For course assignments hosted on GitHub Pages, **Solution 1 (HashRouter)** is almost always the standard and accepted approach because it eliminates server routing configuration hurdles entirely with a single tag swap.

User prompt: Please give me a simple example of a HashRouter and how it is different from the basic router that we use? Also, is the implementation the same but the function different?

Response: The Core Difference: How URLs Are Handled

- **Basic Router (BrowserRouter):** Uses the browser's native HTML5 History API (`pushState`).
 - **URL format:** [`https://username.github.io/my-app/details/toi-286-b`] (`https://username.github.io/my-app/details/toi-286-b`)
 - **What happens on reload:** The browser sends a request to the server asking for the file at `/details/toi-286-b`. Because GitHub Pages has no file there, it returns a **404 Not Found**.
- **Hash Router (HashRouter):** Uses the URL hash (`#`) fragment.
 - **URL format:** [`https://username.github.io/my-app/#/details/toi-286-b`] (`https://username.github.io/my-app/#/details/toi-286-b`)
 - **What happens on reload:** Browsers never send anything after the `#` to the web server. The browser requests only [`https://username.github.io/my-app/`] (`https://username.github.io/my-app/`) (which is your `index.html`), loads it with a **200 OK**, and React reads everything after the `#` to display the right slide.

Is the Implementation the Same?

Yes. The inner implementation is virtually identical.

- You do **not** need to change `<Routes>`, `<Route>`, `<Link>`, `useParams`, or `useNavigate`.
- All your path strings (e.g., `to="/details/..."` or `path="/details/:id"`) stay exactly the same.
- The only difference is the outer wrapper component you import and render around your app.

Minimal Example Comparison

With BrowserRouter (Standard)

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Search</Link>
        <Link to="/gallery">Gallery</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Search />} />
        <Route path="/gallery" element={<Gallery />} />
        <Route path="/details/:id" element={<DetailView />} />
      </Routes>
    </BrowserRouter>
  );
}
```

With HashRouter (GitHub Pages Fix)

You only swap `BrowserRouter` for `HashRouter`:

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { HashRouter, Routes, Route, Link } from 'react-router-dom';

function App() {
  return (
    <HashRouter>
      <nav>
        { /* Notice: Links still point to normal paths; React Router adds the '#' automatically */ }
        <Link to="/">Search</Link>
        <Link to="/gallery">Gallery</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Search />} />
        <Route path="/gallery" element={<Gallery />} />
        <Route path="/details/:id" element={<DetailView />} />
      </Routes>
    </HashRouter>
  );
}
```

Everything else inside your components—including your `prev_img` and `next_img` links—operates without any modifications.

User prompt: Okay. Thank you for the help.

Response: You're very welcome! Best of luck finishing up the deployment and wrapping up the assignment. Feel free to reach back out anytime if other questions come up!
